Clasificación de dígitos MNIST con un perceptrón multicapa en Scikit-learn

Tarea: Construye un modelo de Perceptrón Multicapa (MLP) utilizando la clase MLPClassifier de Scikit-learn para clasificar las imágenes del dataset MNIST.

1. Importación de librerías

Primero importamos las librerías necesarias para trabajar con el dataset MNIST y construir el modelo de Perceptrón Multicapa (MLP).

Utilizaremos:

- `sklearn` para el modelo de clasificación.
- `numpy` para operaciones numéricas.
- `matplotlib` para visualizar datos.
- `confusion_matrix` y `accuracy_score` para evaluar el modelo.

2. Carga del dataset MNIST

Cargamos el dataset MNIST utilizando `fetch_openml`.

Este dataset contiene:

- 70.000 imágenes de dígitos escritos a mano
- Cada imagen tiene tamaño **28x28 píxeles**
- Cada píxel tiene un valor entre **0 y 255**

[2]:
```
mnist = fetch_openml('mnist_784', version=1)

X = mnist.data.to_numpy()
y = mnist.target.astype(int)

print("Forma de X:", X.shape)
print("Forma de y:", y.shape)

print("Tamaño de cada imagen:", X.shape[1])

print("Valor mínimo de los píxeles:", X.min())
print("Valor máximo de los píxeles:", X.max())
```

```
Forma de X: (70000, 784)
Forma de y: (70000,)
Tamaño de cada imagen: 784
Valor mínimo de los píxeles: 0
Valor máximo de los píxeles: 255
```

3. Preprocesamiento de los datos

En imagenes en 8 bits como las de MNIST, cada pixel puede tomar un valor entre 0 y 255, donde 0 representa el color negro y 255 representa el color blanco. Al normalizar los datos, convertimos estos valores a un rango entre 0 y 1, lo que facilita el entrenamiento de la red neuronal.

[3]:
```
X = X / X.max().astype(np.float32)
```

4. División del dataset

Usamos train_test_split para dividir 20% test y 80% train

[4]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print("Datos de entrenamiento:", X_train.shape)
print("Datos de prueba:", X_test.shape)
```

```
Datos de entrenamiento: (56000, 784)
Datos de prueba: (14000, 784)
```

5. Construcción y entrenamiento del modelo MLP

Creamos un modelo de **Perceptrón Multicapa (MLP)** utilizando `MLPClassifier`.

Configuración inicial del modelo:

- Capas ocultas: `(128, 64)`
- Función de activación: `relu`
- Optimizador: `adam`
- Número máximo de iteraciones: `20`

Esto significa:

Primera capa de entrada: 784 neuronas - píxeles, sin pesos propios
Primera capa oculta: 128 neuronas - se transforman las 784 features en 128 representaciones más abstractas, con pesos y bias
Capa de activación: ReLu, pone valores negativos en 0 y positivos los mantiene igual.
Segunda capa oculta: 64 neuronas – sobre la salida anterior, se hace lo mismo que antes.
Capa de salida: 10 neuronas – porque tiene 10 clases (números del 0 al 9)

Utiliza 'adam' como optimizador: estándar que converge rápido y funciona bien sin mucho tuning
Utiliza 20 como parámetro de max_iter, que son las epoch.
Y random_state en 42 que es estándar para fijar la inicializacion aleatoria y consistencia de resultados para hacer el entrenamiento reproducible sin depender de aleatoriedad.

784 entradas –> 128 neuronas –> 64 neuronas –> 10 salidas (dígitos 0-9)
Pixeles –> bordes –> formas –> patrones –> dígitos

(Luego muestro como entreno el modelo con diferentes configuraciones de hiperparámetros y hago sus predicciones en un bucle)

[5]:
```
mlp = MLPClassifier(
    hidden_layer_sizes=(128, 64),
    activation='relu',
```

```
    solver='adam',
    max_iter=20,
    random_state=42
)

mlp.fit(X_train, y_train)
```

[5]:
```
MLPClassifier(hidden_layer_sizes=(128, 64), max_iter=20, random_state=42)
```

6. Predicción

Usamos el modelo entrenado para predecir las etiquetas del conjunto de test.

[6]:
```
y_pred = mlp.predict(X_test)
```

7. Evaluación del modelo

Evaluamos el rendimiento del modelo utilizando la métrica **Accuracy**: el porcentaje de predicciones correctas sobre el conjunto de test.

[7]:
```
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy del modelo:", accuracy)
```

```
Accuracy del modelo: 0.9724285714285714
```

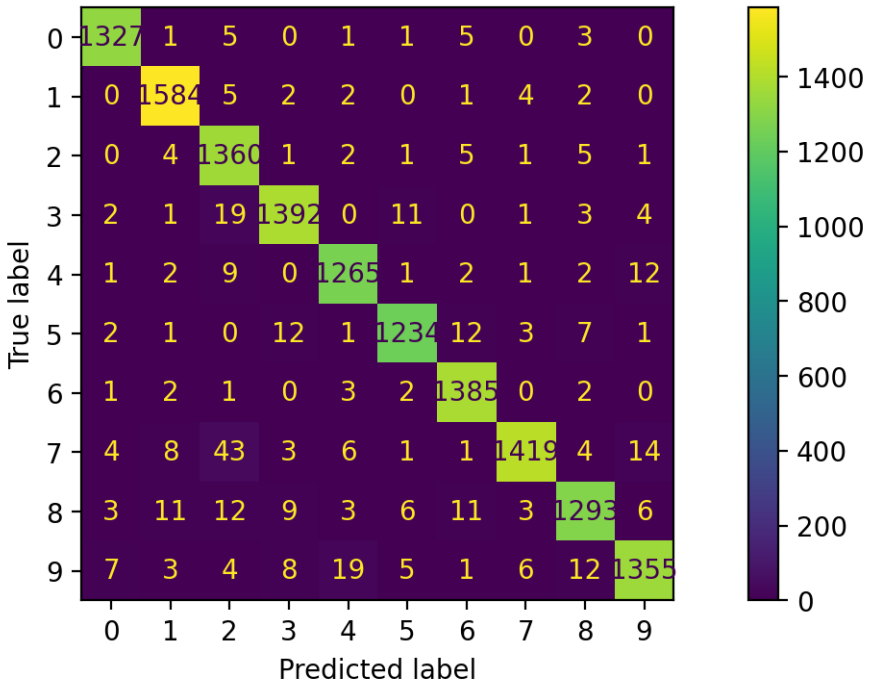
8. Matriz de confusión

La matriz de confusión muestra qué dígitos se clasifican correctamente y cuáles se confunden entre sí.

[8]:
```
cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()

plt.show()
```



9. Prueba con diferentes hiperparámetros

Para analizar cómo afectan los hiperparámetros al rendimiento, probamos diferentes configuraciones de capas ocultas manteniendo el resto de parámetros constantes.

[9]:
```
configuraciones = [
    (50,),
    (100,),
    (128,64),
    (256,128)
]

for config in configuraciones:

    modelo = MLPClassifier(
        hidden_layer_sizes=config,
        activation='relu',
        solver='adam',
        max_iter=20,
        random_state=42
    )

    modelo.fit(X_train, y_train)

    pred = modelo.predict(X_test)

    acc = accuracy_score(y_test, pred)
```

```
    print(f"Capas ocultas {config} -> Accuracy: {acc}")
```

```
Capas ocultas (50,) -> Accuracy: 0.9688571428571429
Capas ocultas (100,) -> Accuracy: 0.9746428571428571
Capas ocultas (128, 64) -> Accuracy: 0.9724285714285714
Capas ocultas (256, 128) -> Accuracy: 0.9795714285714285
```

Resultados

Podemos observar que:

- La arquitectura `(128, 64)` obtiene un **97.24 %**, ligeramente inferior a una sola capa de 100 neuronas (97.46 %). Añadir profundidad no garantiza mejora si el número total de neuronas es menor.
- La mejor configuración es `(256, 128)` con un **97.96 %**, lo que sugiere que un mayor número total de neuronas permite aprender representaciones más ricas.
- En conjunto, todas las configuraciones se mantienen entre el 96.9 % y el 97.9 %, lo que apunta a un techo de rendimiento del MLP en esta tarea. Las CNN de los notebooks de PyTorch y Keras superan este límite al aprovechar la estructura espacial de las imágenes.

Conclusiones

El MLP de Scikit-learn clasifica MNIST con ~97.2–97.9% de accuracy según la arquitectura. Su ventaja es la simplicidad: apenas unos parámetros para definir el modelo y `fit()` para entrenarlo. Su limitación estructural es tratar cada píxel como una característica independiente (vector de 784 valores), ignorando completamente la estructura espacial de la imagen: bordes, curvas y formas no son explotados como tales.

La comparación de arquitecturas (Sección 9) confirma rendimientos decrecientes al añadir capas: el techo práctico del MLP en MNIST ronda el 97.9%, un límite que las CNN de los notebooks de PyTorch y Keras superan ampliamente al explotar la localidad espacial de los píxeles.